

Numerical Simulation of Projectile Motion with Quadratic Air Resistance

Zachary Cichawa 252593

1. Introduction

This project studies projectile motion with air resistance using numerical simulation. The projectile is represented as a moving spherical object, such as a tennis ball, baseball, or steel ball. The motion is affected by gravity and by drag, which acts opposite to the direction of motion.

The main numerical methods used in the project are Euler, RK2, and RK4. These methods are used because the model is written as a system of ordinary differential equations, and the acceleration changes continuously during flight because drag depends on the current speed. This makes the problem suitable for numerical ODE solvers rather than only basic constant-acceleration formulas.

The project has two main goals. The first goal is to compare how different physical objects behave under the same launch conditions. The second goal is to compare how accurately Euler, RK2, and RK4 approximate the same projectile motion problem at different step sizes.

2. Problem statement and theoretical background

The problem is to simulate the trajectory of a projectile launched with a fixed initial speed, launch angle, and initial height. The projectile moves in two dimensions: horizontal distance x and vertical height y .

The state vector used in the simulation is:

```
state = [x, y, vx, vy]
```

Here, x is horizontal position, y is height, v_x is horizontal velocity, and v_y is vertical velocity. At any moment, these four values describe the projectile state.

2.1 Air resistance model

Air resistance is modeled using quadratic drag. This means that the magnitude of the drag force depends on the square of the current speed. The drag force magnitude is:

```
F_drag = 0.5 * rho * Cd * A * speed^2
```

In this formula, ρ is air density, C_d is the drag coefficient, A is the cross-sectional area of the object, and $speed$ is the total speed of the projectile. For a spherical object, the cross-sectional area is:

```
A = pi * r^2
```

The drag force must be converted into acceleration because the numerical solvers update velocity using acceleration. Newton's second law gives $F = m \cdot a$, so acceleration is force divided by mass. The constant part of the drag acceleration is grouped into the drag parameter k :

```
k = (rho * Cd * A) / (2m)
```

For one object, k is constant because mass, radius, drag coefficient, and air density are treated as constant. However, the actual drag acceleration is recalculated at every solver step because it also depends on the current speed and velocity components.

2.2 Differential equations

The equations of motion used in the project are:

$$\begin{aligned}x' &= vx \\y' &= vy \\vx' &= -k * speed * vx \\vy' &= -g - k * speed * vy\end{aligned}$$

The first two equations mean that position changes according to velocity. The third equation describes horizontal deceleration due to drag. The fourth equation describes vertical acceleration due to gravity and vertical drag. The total speed is calculated as:

$$speed = \sqrt{vx^2 + vy^2}$$

Gravity appears only in the vertical equation because gravity acts downward, not horizontally. Drag appears in both velocity equations because drag acts opposite to the full direction of motion.

2.3 Initial conditions

The notebook uses the following launch conditions for the experiments:

- Initial speed: 30 m/s
- Launch angle: 40 degrees
- Initial height: 2 m
- Gravity: 9.81 m/s²
- Air density: 1.225 kg/m³

The initial speed is split into horizontal and vertical velocity components using trigonometry:

$$\begin{aligned}vx0 &= speed * \cos(angle) \\vy0 &= speed * \sin(angle)\end{aligned}$$

2.4 Convergence criteria

The simulation stops when the projectile reaches the ground, which is defined as $y \leq 0$. The solvers also use a maximum time value t_{end} as a safety limit. For error analysis, the tested methods are compared with a reference solution computed using RK4 with a very small step size $h = 0.001$. Convergence is evaluated by checking whether the error decreases as the step size becomes smaller.

3. Numerical methods

The same physical ODE system is solved using three numerical methods: Euler, RK2, and RK4. All three methods advance the state from time t_n to $t_n + h$, where h is the step size.

3.1 Euler method

Euler is the simplest method. It uses only the derivative at the beginning of the step:

```

k1 = f(t_n, y_n)
y_(n+1) = y_n + h*k1

```

Euler is computationally cheap but usually has the largest error because it assumes the current slope stays valid over the whole step.

3.2 RK2 method

RK2 first estimates the slope at the beginning of the step, then estimates a midpoint state and uses the midpoint slope to update the solution:

```

k1 = f(t_n, y_n)
k2 = f(t_n + h/2, y_n + h*k1/2)
y_(n+1) = y_n + h*k2

```

RK2 is more accurate than Euler because it uses information from the middle of the step.

3.3 RK4 method

RK4 uses four slope estimates and combines them using a weighted average:

```

k1 = f(t_n, y_n)
k2 = f(t_n + h/2, y_n + h*k1/2)
k3 = f(t_n + h/2, y_n + h*k2/2)
k4 = f(t_n + h, y_n + h*k3)
y_(n+1) = y_n + (h/6)*(k1 + 2*k2 + 2*k3 + k4)

```

RK4 is the highest-order method used in the project. It requires more derivative evaluations per step, but it is generally expected to give a more accurate approximation than Euler and RK2 for the full solution.

4. Algorithm implementation

Pseudocode of the simulation algorithm

1. Select projectile object.
2. Read mass, radius, and drag coefficient.
3. Compute cross-sectional area $A = \pi * r^2$.
4. Compute drag parameter $k = (\rho * C_d * A) / (2m)$.
5. Create initial state $[x_0, y_0, vx_0, vy_0]$ from launch speed, angle, and initial height.
6. Choose numerical solver: Euler, RK2, or RK4.
7. While final time is not reached:
 - a. Compute current speed $= \sqrt{vx^2 + vy^2}$.
 - b. Compute derivatives:

$$x' = vx$$

$$y' = vy$$

$$v_x' = -k * \text{speed} * v_x$$

$$v_y' = -g - k * \text{speed} * v_y$$

c. Use selected solver to calculate next state.

d. Store time and state.

e. Stop if $y \leq 0$.

8. If the last point is below ground, interpolate the landing point at $y = 0$.

9. Calculate range, maximum height, flight time, and final speed.

10. Compare numerical methods with the reference solution and calculate errors.

The final project implementation is divided into two Python source files and one notebook.

- projectile.py defines the physical model, object parameters, drag parameter calculation, initial state, ground-hit condition, and trajectory summary.
- solvers.py defines the generic numerical ODE solvers: Euler, RK2, and RK4.
- 01.ipynb runs the experiments, calculates errors, creates tables and plots, and presents the results.

4.1 Physical model implementation

The most important physical function is `projectile_with_drag(t, state, k)`. It receives the current time, current projectile state, and object drag parameter, then returns the current rates of change $[x', y', v_x', v_y']$.

```
speed = sqrt(vx^2 + vy^2)
dxdt = vx
dydt = vy
dvxdt = -k * speed * vx
dvydt = -gravity - k * speed * vy
```

The object configuration function calculates the cross-sectional area and the drag parameter k for each object. The trajectory summary function calculates range, maximum height, flight time, and final speed. If the final solver point is below ground level, linear interpolation is used between the last two points to estimate the landing point more accurately.

4.2 Solver implementation

Each solver follows the same general pattern: initialize time and state arrays, repeatedly calculate derivative estimates, update the state, store the result, and stop when the projectile reaches the ground.

```
initialize t = t0 and y = y0
while t < t_end:
    compute derivative estimates using f(t, y)
    compute next state using Euler, RK2, or RK4
    store next time and state
    if y <= 0: stop
```

The solvers also count the number of function evaluations. Euler uses one derivative evaluation per step, RK2 uses two, and RK4 uses four. This count is used as an algorithmic measure of computational cost.

4.3 Notebook experiments

The notebook performs two experiments. First, it compares the tennis ball, baseball, and steel ball using the same launch conditions and RK4 with $h = 0.01$. Second, it fixes the object to the tennis ball and compares Euler, RK2, and RK4 for step sizes $h = 0.1, 0.05, 0.02$, and 0.01 . A fine-step RK4 solution with $h = 0.001$ is used as the reference solution.

5. Results and error analysis

Table 1. Object comparison results using RK4 with $h = 0.01$.

Object	Mass kg	Radius m	Cd	k	Range m	Max height m	Flight time s	Final speed m/s
Tennis Ball	0.057	0.0335	0.55	0.020837	42.568	13.777	3.224	15.879
Baseball	0.145	0.0366	0.35	0.006222	66.618	17.724	3.694	22.331
Steel Ball	0.500	0.0250	0.47	0.001130	86.169	20.227	3.960	28.390

The object comparison shows that the tennis ball has the largest drag parameter and the shortest range. The steel ball has the smallest drag parameter and travels the farthest. This is expected because a smaller drag effect allows the object to keep more of its velocity during flight.

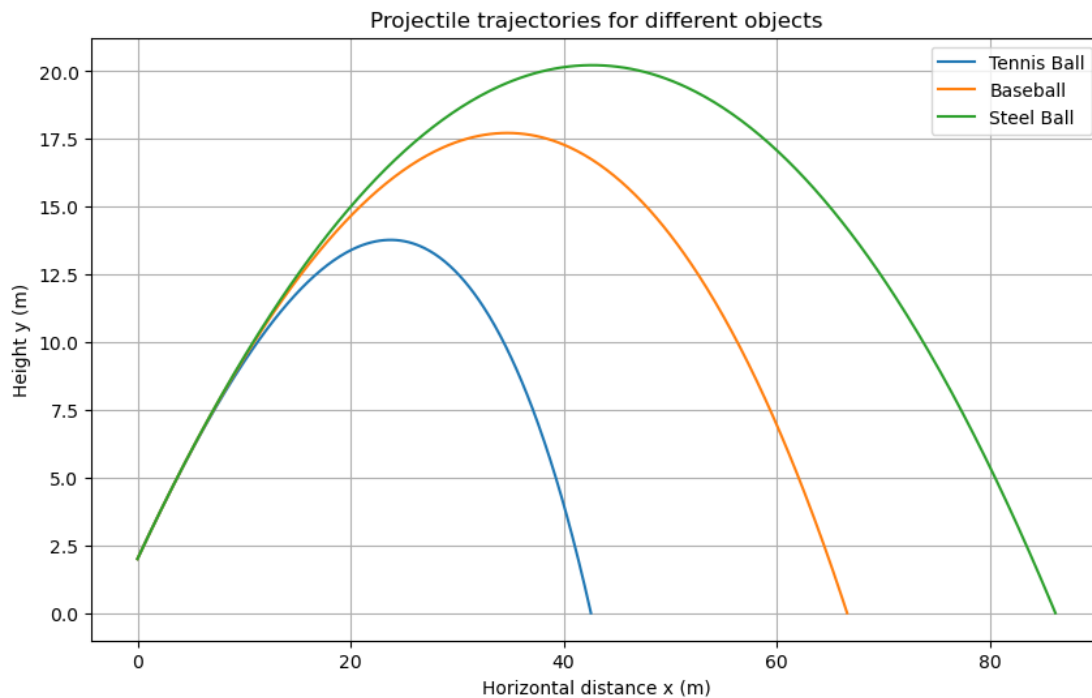


Figure 1. Projectile trajectories for different objects.

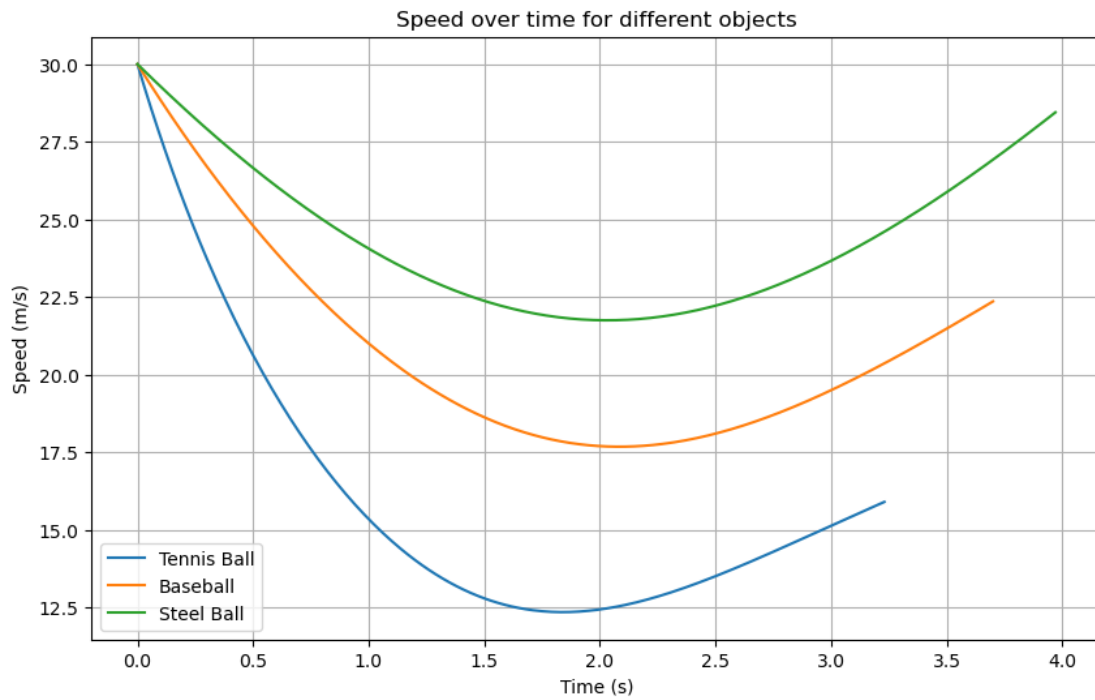


Figure 2. Speed over time for different objects.

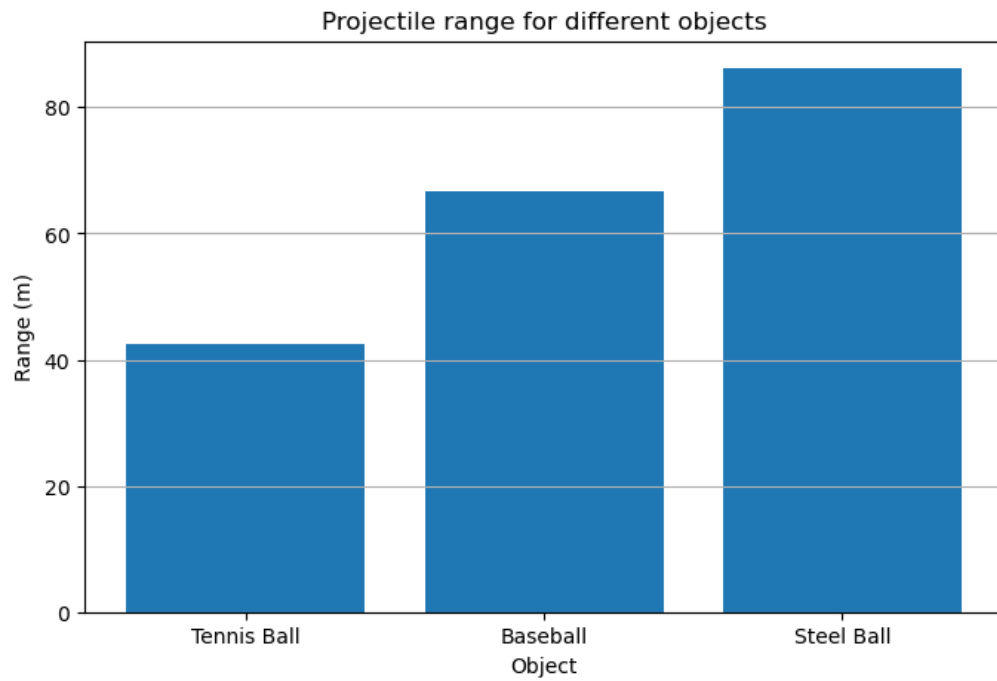


Figure 3. Projectile range for different objects.

5.1 Numerical method comparison

The numerical method comparison uses the tennis ball as the fixed object. Each method is compared with the fine-step RK4 reference solution. The errors are calculated for range, maximum height, flight time, and the whole trajectory position.

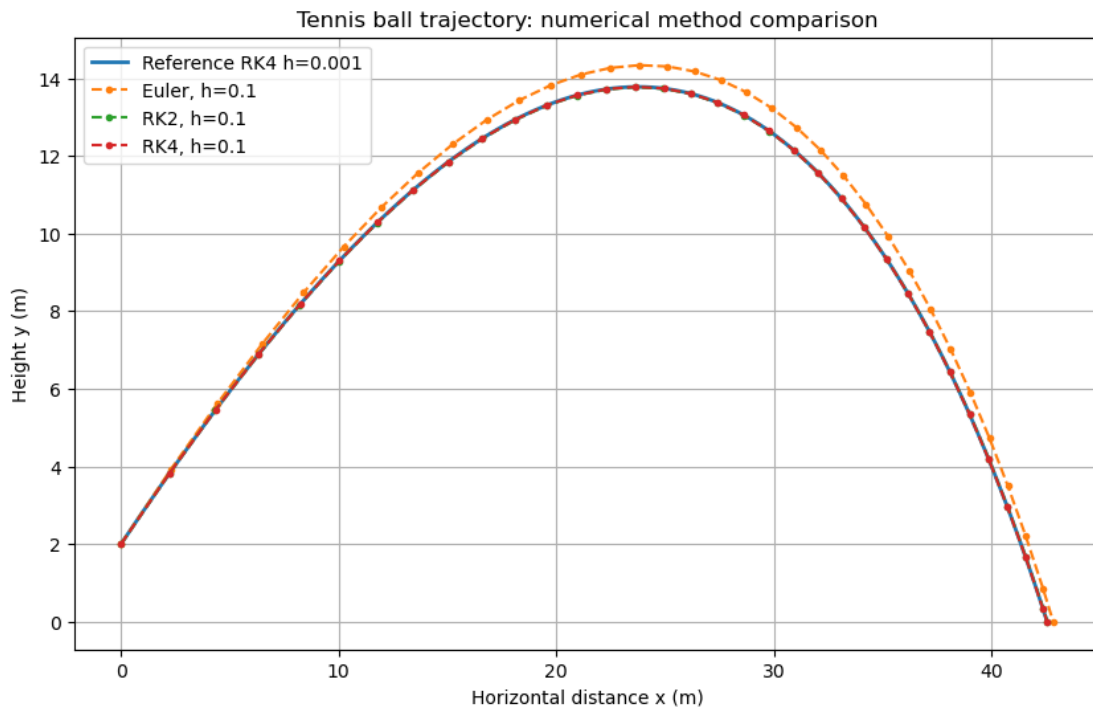


Figure 4. Tennis ball trajectory comparison for Euler, RK2, RK4, and the reference solution.

Table 2. Error summary for tennis ball method comparison.

Solver	h	Range error m	Relative range error	Max height error m	Flight time error s	MSE position
Euler	0.10	0.280593	0.6592%	0.557425	3.644464e-02	1.428030e-01
Euler	0.05	0.149433	0.3510%	0.276309	1.871477e-02	3.412700e-02
Euler	0.02	0.061407	0.1443%	0.109829	7.567509e-03	5.532000e-03
Euler	0.01	0.031042	0.0729%	0.055044	3.801942e-03	1.369000e-03
RK2	0.10	0.008119	0.0191%	0.006140	1.455614e-04	2.174000e-02
RK2	0.05	0.001457	0.0034%	0.001640	6.119532e-07	1.299000e-03
RK2	0.02	0.000333	0.0008%	0.000485	6.917923e-06	1.990000e-04
RK2	0.01	0.000060	0.0001%	0.000080	1.596426e-07	1.400000e-05
RK4	0.10	0.004992	0.0117%	0.000288	3.385163e-04	2.175500e-02
RK4	0.05	0.001739	0.0041%	0.000276	1.183524e-04	1.298000e-03
RK4	0.02	0.000168	0.0004%	0.000276	1.144515e-05	1.990000e-04
RK4	0.01	0.000065	0.0002%	0.000029	4.402474e-06	1.400000e-05

5.2 Error measures

Range error measures the difference between the computed landing distance and the reference landing distance. Relative range error divides the range error by the reference range, so the error can be understood as a proportion of the total distance.

Maximum height error measures the difference between the computed peak height and the reference peak height. Flight time error measures the difference between the computed landing time and the reference landing time.

Mean squared position error, abbreviated as MSE, compares the whole trajectory shape. It compares the computed x and y positions with the reference x and y positions at the same time points, squares the differences, and averages them. A smaller MSE means the full trajectory is closer to the reference solution.

Interpolation is used in two places. Landing interpolation estimates the point where $y = 0$ when the solver steps from above the ground to below the ground. This affects range and flight time. Reference interpolation is used for MSE, where the fine-step reference trajectory is interpolated onto the tested method's time grid so that both trajectories can be compared at the same times.

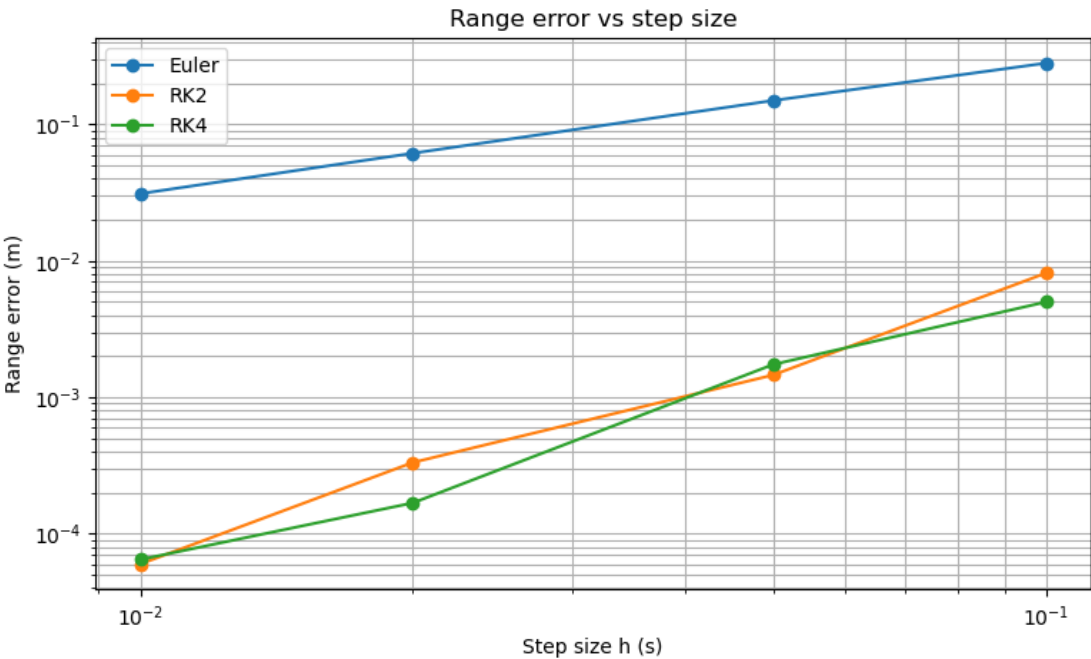


Figure 5. Range error vs step size.

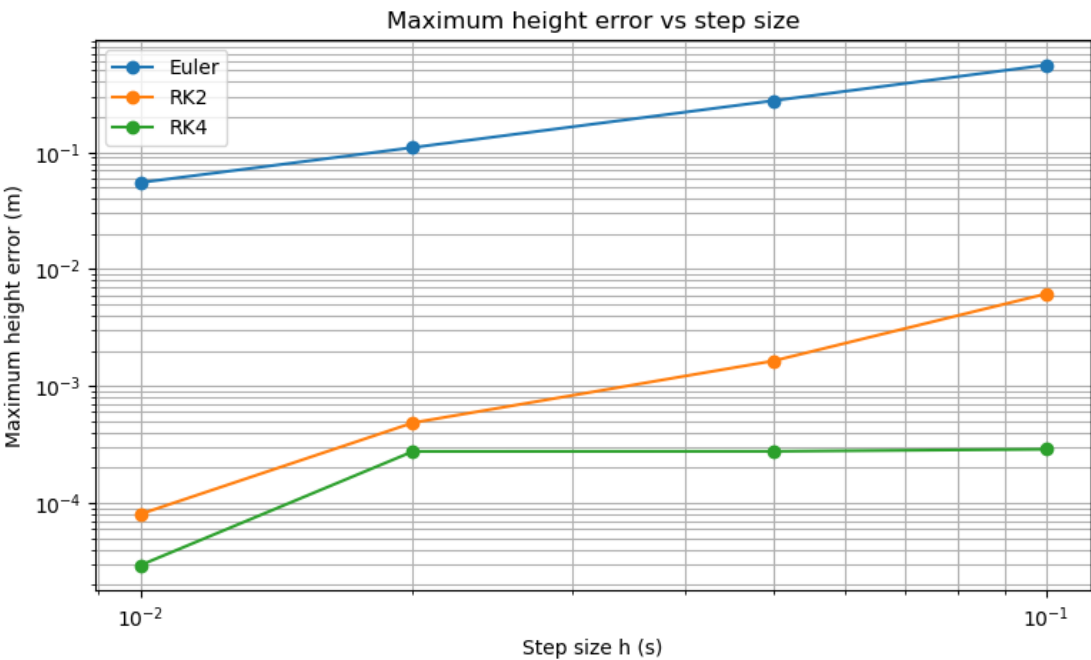


Figure 6. Maximum height error vs step size.

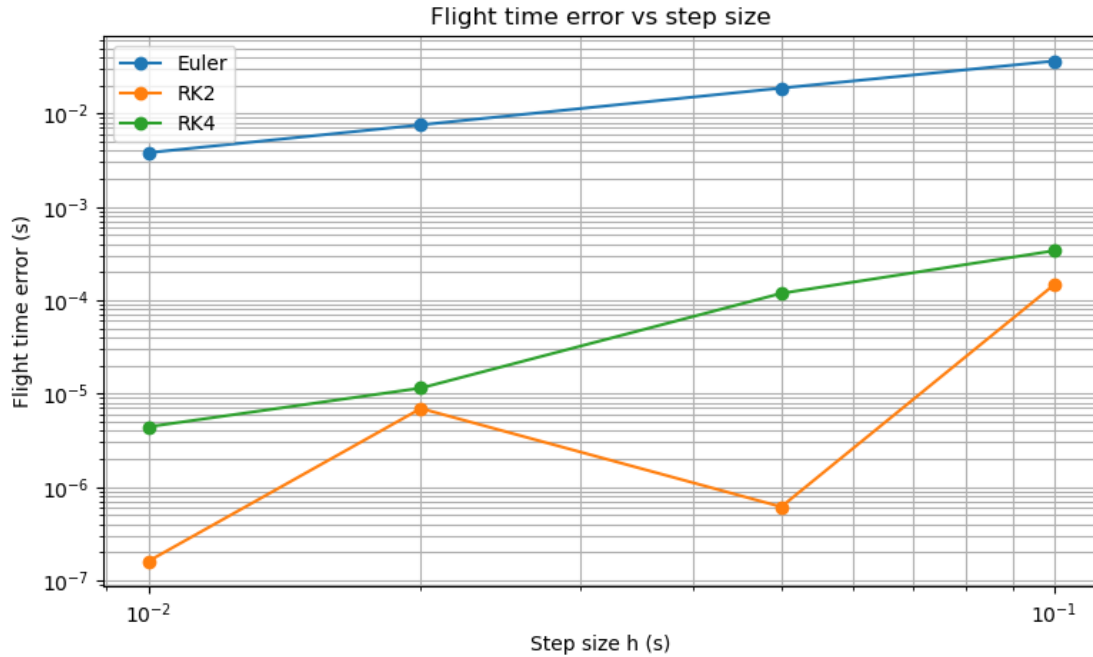


Figure 7. Flight time error vs step size.

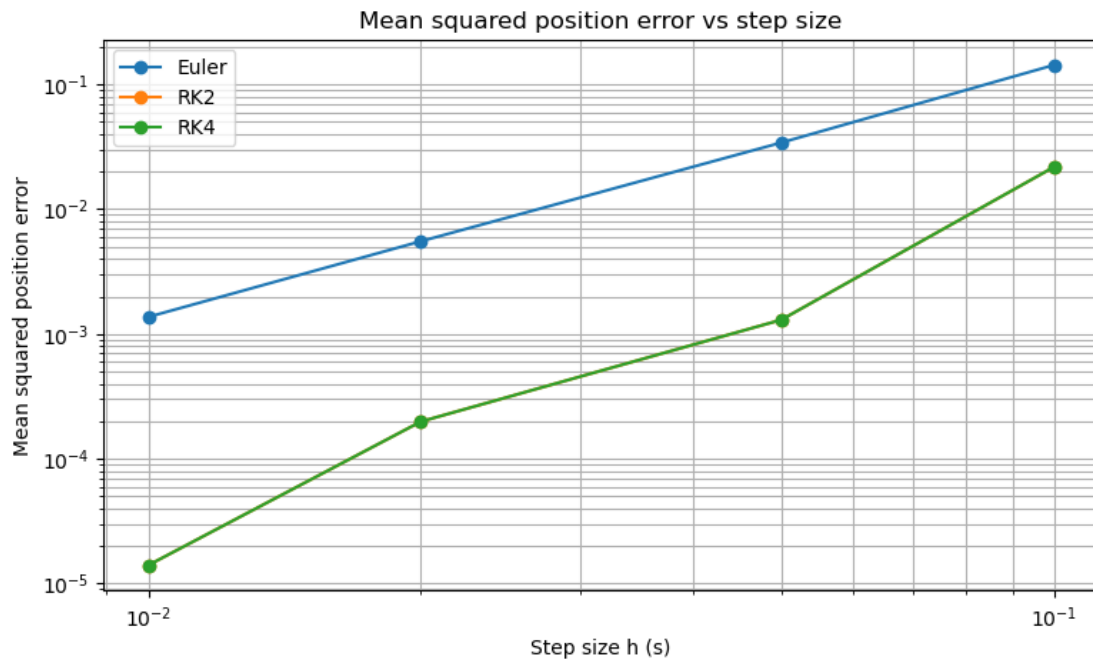


Figure 8. Mean squared position error vs step size.

5.3 Interpretation of numerical errors

The error results show that Euler produces the largest errors for the tested step sizes. This is expected because Euler uses only one derivative estimate per step. RK2 and RK4 are much closer to the reference solution.

The results also show that the errors generally decrease as the step size decreases. This confirms the expected convergence behavior of the numerical methods.

Some individual scalar outputs do not produce a perfect ranking where RK4 is always lower than RK2. For example, RK2 may be slightly closer for range or flight time at some step sizes. This can happen because range and flight time are derived from the final ground-crossing point and are affected by interpolation and small error cancellation. The important overall conclusion is that Euler is clearly worse, while RK2 and RK4 are both close to the reference.

6. Discussion

The project demonstrates why numerical methods are useful for projectile motion with air resistance. Without drag, standard projectile formulas can be used because acceleration is constant. With quadratic drag, acceleration changes during the motion because it depends on the current speed. Therefore, the trajectory is solved step by step using ODE solvers.

The object comparison shows the importance of physical parameters. The tennis ball is affected most strongly by air resistance and therefore has the shortest range. The steel ball is least affected by drag and travels the farthest. This agrees with the drag parameter formula, where a larger area or drag coefficient increases drag and a larger mass reduces the acceleration caused by drag.

The numerical method comparison shows the accuracy-cost tradeoff. Euler is simple and cheap but less accurate. RK2 and RK4 use more derivative evaluations per step and produce results much closer to the reference. RK4 is the highest-order method in the project, but some final summary quantities can occasionally make RK2 appear slightly closer because of interpolation and error cancellation.

6.1 Model limitations

The model is simplified. It treats each projectile as a sphere, assumes constant air density, and uses a constant drag coefficient. It does not include wind, spin, lift, changing air conditions, or deformation of the object. These assumptions keep the model suitable for a numerical methods project, but they limit its realism compared with a full aerodynamic simulation.

7. Conclusion

This project simulated projectile motion with quadratic air resistance using a system of ordinary differential equations. The physical model was implemented in `projectile.py`, the numerical solvers were implemented in `solvers.py`, and the experiments were performed in the notebook.

The object comparison showed that physical properties strongly affect projectile motion. The tennis ball had the largest drag effect and shortest range, while the steel ball was least affected by air resistance and traveled the farthest.

The numerical method comparison showed that Euler produced the largest errors, especially for larger step sizes. RK2 and RK4 were much closer to the fine-step RK4 reference solution. As the step size decreased, the errors generally decreased, confirming convergence. Overall, the project shows how numerical ODE solvers can be used to simulate physical systems where acceleration changes during motion because of velocity-dependent air resistance.